CRICKETEC · EXPERIMENT 3

Frame-Count Sensitivity — Windows Guide

How does the **number of sampled frames** affect accuracy and computational cost? Everything is held fixed (ResNet-34 backbone, GRU head, Uniform sampling, identical settings and seed) and only the frame count varies: **8, 16, 24, 32**. Each is trained, scored on the test set, and timed for inference latency and GPU memory. Uses the **same project folder** as Experiments 1–2 — just one extra script.

---

0. Prerequisite — the Experiment 1 setup

This assumes the one-time setup from the Experiment 1 guide is already done: Python 3.11, the **GPU** build of PyTorch, the dependencies, and the unzipped `cricketec\` folder with the dataset inside. This experiment trains its own models, so it needs the dataset (already in the folder) but **not** the checkpoints.

1. Add the script

Copy `experiment_frame_sensitivity.py` into the `cricketec\` folder, next to the other scripts. Open PowerShell there:

```
cd C:\cricketec
```

2. First run needs internet (one-time)

Like Experiment 2, the first run downloads the ResNet-34 ImageNet starting weights (~85 MB) once, then caches them. If this machine already ran Experiment 2, the weights are cached and no download happens.

3. Quick check first (2 epochs)

Confirm the whole pipeline works on this machine before the long run — trains only 2 epochs, finishes in a few minutes:

```
python experiment_frame_sensitivity.py --epochs 2 --device cuda
```

It should print `Device: cuda (…)`, train all four, and write the results CSV + two PNGs into `experiment3\`.

4. Full run

Train all four frame counts for the full 40 epochs each:

```
python experiment_frame_sensitivity.py --device cuda
```

> **This trains four models from scratch**, so it takes a while — the 32-frame config is the slowest. Plan for several hours on a laptop GPU (it prints a per-config time). You can split the work and resume later with, e.g., `--configs E3-A E3-B`; each config's best checkpoint is saved as it goes.

The four configurations (ResNet-34 + GRU + Uniform sampling, fixed seed = 42; only the frame count changes):

ID	Frames
E3-A	8
E3-B	16
E3-C	24
E3-D	32

For each config it trains, picks the epoch with the best validation accuracy, scores that checkpoint on the test set, and then times single-clip inference and records peak GPU memory — so accuracy and cost are both measured under the same protocol.

5. Results (in `experiment3\`)

File	Contents
`frame_sensitivity_results.csv` / `.json`	one row per frame count: test accuracy, macro P/R/F1, per-clip latency (ms), peak GPU memory (MB), best epoch, train time
`frame_count_vs_accuracy.png`	frames (8→32) vs test accuracy and macro-F1
`frame_count_vs_latency.png`	frames vs per-clip inference latency and peak GPU memory
`weights\E3-A_best.pth … E3-D_best.pth`	the best-val checkpoint kept for each config

6. If something goes wrong

Problem	Fix
`CUDA available: False` / very slow	CPU-only torch — reinstall the GPU build with the `--index-url` line from the Experiment 1 guide.
Latency / memory columns are blank	Those are GPU metrics — they only populate on `--device cuda`. On CPU, memory is left empty by design.
First run fails downloading weights	No internet / proxy blocked it — connect to the internet for the first run.
Hangs at the very start	Windows DataLoader — add `--workers 0` to the command.
Out of GPU memory (most likely on 32 frames)	Run the big one alone with `--configs E3-D`, or lower `BATCH_SIZE` near the top of the script.